

Code-Layout, Readability and Reusability

Date	29 June 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP 8 styling (4-space indentation) is strictly followed in Python backend app.py, and consistent spacing is maintained in script.js and frontend templates.
2	Proper File Structure	Files and folders are logically organized	Yes	Clear physical separation of concerns is maintained with Dataset/, Training/ (ML models), Flask/ (backend/frontend UI)
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variable names mirror their schema parameters (e.g., ApplicantIncome, Credit_History, LoanAmount) and runtime states (e.g., scaled_features, confidence_percent, risk_band).
4	Function / Method Names	Functions are descriptively named	Yes	Clear verb-noun naming conventions are utilized for handlers: backend functions parse_and_validate_form, get_risk_band and frontend functions updateProgress, updateBurden, updateLiveHint.
5	Code Comments	Inline and block comments explain logic	Yes	Annotations document model loading, user input schemas/rules, custom data scaling, local storage preset settings, and DOM validation events.
6	Modular Design	Code is split into reusable functions/modules	Yes	Separation of duties is maintained. Backend routes invoke modular validator functions, and frontend components isolate visual burden computations from raw model calls.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Production files are clean and imports are minimized to required dependencies (e.g. numpy, pickle, math). Standard scripts use event mapping to avoid redundant element listener

				declarations.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Python backend employs targeted try-except blocks to catch formatting/calculation exceptions in app.py:L131-134 while client-side scripts wrap localStorage logic in try-catch blocks in script.js:L185-194.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Input Schema & Validator (parse_and_validate_form)	Python / Flask	Validates user input request forms against standard domain constraints (FIELD_RULES) in POST routes before pipeline feeding.	High
2	Model Scaler Module (scale.pkl)	Python / Scikit-Learn	Serialized data scaler trained in Loan_Prediction_using_ML.ipynb and loaded in Flask backend for live feature transformations.	High
3	Risk Assessment Handler (get_risk_band)	Python / Flask	Translates continuous raw probability estimates into categorical risk bands for output visualization.	Medium
4	Dynamic Progress & Burden Indicators (updateProgress, updateBurden)	JavaScript / ES6	Recalculates real-time loan-to-income and application completion states on multiple dynamic form inputs.	Medium
5	State Persistence Layer (Draft Manager)	JavaScript / localStorage API	Local storage utility methods that serialize/deserialize theme selections and draft form parameters.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Excellent layout separating raw data, exploratory training notebooks, web-app services, and cloud deployment pipelines.
Readability	5	Clean variable declaration syntax matching the core data schema, proper functional headers, and strict adherence to standard styling guidelines.
Reusability	4.5	Highly modular form input validation framework and standalone client-side logic scripts. Machine learning models and preprocessing scalers are exported to generic pickle files.
Documentation / Comments	4.8	High-quality inline code documentation describing pipeline transformations, UI interactive hooks, and dataset constraints.
Overall Score	4.8/5	The system showcases a very professional, well-structured, and highly readable architecture, meeting software engineering best practices for machine learning microservices.